

Document Version: 1.0 Status: Final for Implementation Based On: Tech Stack Justification v1.0, System Architecture v1.0, Database Design v1.0, Folder Structure v1.1, API Design v1.0

1. Purpose

This document specifies the internal technical design of the `ai-pipeline/sentinelcut_ai/` module — the code that actually runs inside the Celery/RQ worker processes when a `POST /files/{id}/process` or `POST /files/{id}/regenerate` request is dequeued. Every prior document justified *what* to build and *why* (Tech Stack), *where it sits* in the system (Architecture), *what it persists* (Database Design), and *what triggers it* (API Design). This document defines *how each stage actually works*: exact algorithms, interfaces, data structures, and pseudocode, so a developer can implement `ai-pipeline/` directly from this spec without further design decisions.

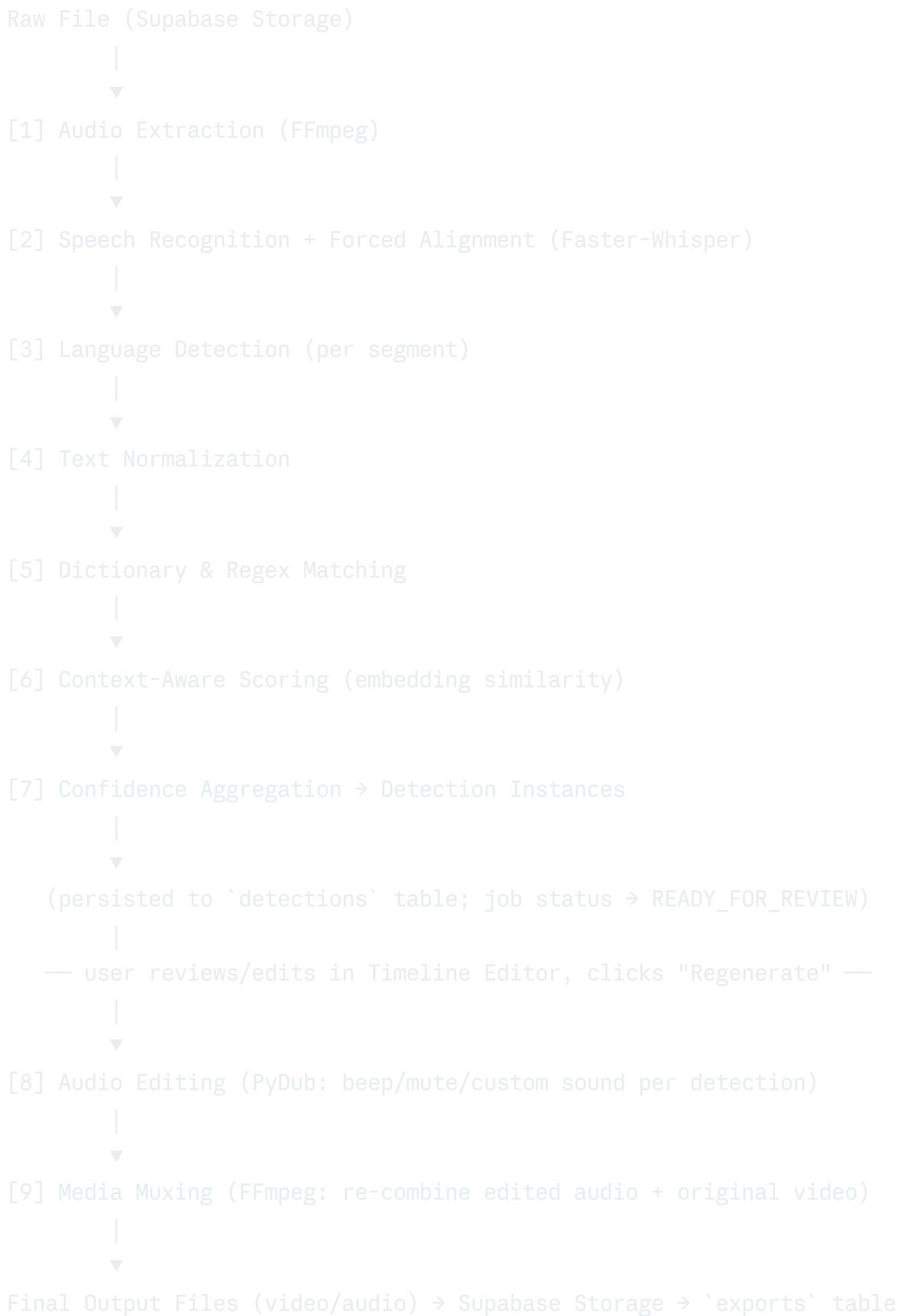
This document is the direct technical precursor to implementation and is written for the engineer(s) who will build the modules under `ai-pipeline/sentinelcut_ai/`.

2. Scope

In scope: the nine internal stages that run inside `processing_pipeline.py`, from raw file to persisted detection instances plus the audio-editing/mux stage that produces the censored output.

Out of scope (covered elsewhere): job queueing/retry orchestration at the Celery level (System Architecture §4.3/§6.1), database persistence mechanics (Database Design), and the REST/WebSocket contract that surfaces pipeline status to the frontend (API Design §12).

3. Pipeline Overview



Stages 1-7 run during `initial_processing` jobs. Stage 4-7 also re-run in isolation if the user manually adds a detection with free text (validated against the normalizer). Stages 8-9 run during both `initial_processing` (first pass, using whatever detections exist at that point) and `regeneration` jobs (using the current, user-edited detection set — this is the "Regenerate" action from PRD §4.7/FR-TLE-08).

Every stage is implemented as a class with a single well-defined entrypoint method, registered behind an interface where the System Architecture document (§7.3) already committed to swappability (`SpeechRecognitionService`, `ProfanityDetectionService`). This document defines those interfaces concretely.

4. Stage 1 — Audio Extraction

Module: `audio_processing/audio_extractor.py`

Responsibility: Given a video or audio file at `storage_path`, produce a normalized mono 16kHz WAV file suitable for ASR input, and populate `duration_seconds` on the `files` row.

Interface:

```
python

class AudioExtractor:
    def extract(self, input_path: str, output_path: str) -> AudioExtractionResult:
        """
        Runs: ffmpeg -i {input_path} -vn -ac 1 -ar 16000 -f wav {output_path}
        Returns duration_seconds, sample_rate, channel_count.
        Raises AudioExtractionError on non-zero ffmpeg exit code or unreadable st
```

Design notes:

- If the input is already audio-only (mp3/wav/aac/flac/m4a), the `-vn` flag is a no-op; the same code path handles both audio and video uploads, avoiding a branch per file type.
- 16kHz mono is the standard input rate for Whisper-family models; extracting at this rate up front avoids a second resampling pass inside the ASR engine.
- The original video container is **never modified** at this stage — only a working-copy audio stream is pulled out. The original video file is preserved untouched for the Stage 9 re-mux, which is what guarantees resolution/frame-rate/metadata preservation (Product Vision constraint, SRS-FR-EXPT-01).
- On extraction failure (corrupted codec, empty audio track), the job is marked `FAILED` with `error_message` populated from the ffmpeg stderr tail, surfaced via the edge case in Architecture §4.3 (Failed ASR / Retry sequence — extraction failures follow the identical retry path as ASR failures).

5. Stage 2 — Speech Recognition (ASR) + Forced Alignment

Module: `asr/speech_recognition_service.py` (interface),

`asr/faster_whisper_engine.py` (implementation), `asr/forced_alignment.py`

Interface (swap point per Architecture §7.3):

```
python

class SpeechRecognitionService(ABC):
    @abstractmethod
    def transcribe(self, audio_path: str, language_hint: str | None = None) -> T:
        """Returns word-level segments: [{text, start, end, confidence}], plus de
```

Faster-Whisper implementation details:

Parameter	Value	Rationale
Model size	<code>medium</code> (default), <code>small</code> as CPU-constrained fallback	Balances Hindi/Hinglish accuracy against CPU latency budget ($\leq 2\times$ real-time, SRS-NFR-PERF-01)
Compute type	<code>int8</code> on CPU, <code>float16</code> on GPU	CTranslate2 quantization; <code>int8</code> is the primary lever for meeting the CPU-first constraint from the Tech Stack doc
<code>word_timestamps</code>	<code>True</code>	Required directly by FR-TLE-01/02
<code>vad_filter</code>	<code>True</code>	Skips silence, reducing hallucinated transcriptions on music-only files (Edge Case table, PRD §7)
<code>beam_size</code>	<code>5</code>	Standard Whisper-family default; higher values traded against latency budget
Chunking	Files > 10 minutes are split into overlapping ~30s windows with 2s overlap, transcribed independently, then stitched by discarding duplicate words in the overlap region based on timestamp continuity	Bounds memory and enables the "chunk long files, show incremental progress" behavior required by the Edge Case table (3-hour lecture)

Forced alignment: Faster-Whisper's native `word_timestamps` already performs a WhisperX-style alignment internally (this is why Faster-Whisper was selected over vanilla Whisper in the Tech Stack doc — see §3.3 point 4). No separate wav2vec2 alignment pass is required for MVP; `forced_alignment.py` is kept as a thin, currently-pass-through module specifically so a dedicated alignment model (e.g., an Indic wav2vec2 checkpoint) can be inserted later without touching `faster_whisper_engine.py` or any calling code — this is the concrete realization of the "swap point" language used in the Architecture document.

Output contract (`TranscriptionResult`):

```
json
{
  "words": [
    {"text": "This", "start": 0.00, "end": 0.18, "confidence": 0.97},
    {"text": "fucking", "start": 1.22, "end": 1.61, "confidence": 0.89}
  ],
  "raw_segments": [{"text": "This movie is fucking amazing", "start": 0.0, "end": 1.61}],
  "language_guess": "en"
}
```

Word-level output feeds Stage 4–7 directly (each word/short n-gram is a matching unit); `raw_segments` feed the `transcript.full_text` field surfaced at `GET /files/{id}/transcript`.

6. Stage 3 — Language Detection

Module: `language_detection/language_detector.py`

Responsibility: Assign one of `english | hindi | hinglish | mixed` per segment (not just per file), satisfying SRS-FR-LANG-01/03.

Algorithm:

1. Take Faster-Whisper's per-segment `language_guess` (Whisper's internal detector) as a first signal — it is reliable for monolingual segments but weak on genuinely code-mixed sentences.
2. Run a lightweight script-detection pass over the transcribed text itself: classify each token as Latin-script or Devanagari-script.
3. Combine signals per segment:
 - All-Latin + Whisper says `en` → `english`
 - All-Devanagari + Whisper says `hi` → `hindi`

- Mixed Latin/Devanagari tokens within one segment, OR Latin-script tokens that match Romanized-Hindi phonetic patterns (checked against the Hinglish slang dictionary from Stage 5) → `(hinglish)`
 - Adjacent segments in the same file disagree on language → file-level `(detected_language)` is set to `(mixed)`, while each segment retains its own individual tag (this is what SRS-FR-LANG-03's code-switching requirement actually needs — segment-level tags, not a single file-level guess)
4. If confidence in step 3 is low (ambiguous script + no dictionary hit), default to `(hinglish)` rather than `(english)`, since Romanized Hindi profanity is the case most likely to be silently missed if defaulted the other way — this directly protects the recall target in SRS-FR-PDE.

The manual override endpoint behavior (SRS-FR-LANG-04, exposed via the frontend as a language dropdown) simply re-invokes Stage 2 with `(language_hint)` forced, then re-runs Stages 3-7; it does not require new pipeline logic beyond honoring the `(language_hint)` parameter already defined in the `(SpeechRecognitionService)` interface above.

7. Stage 4 — Text Normalization

Module: `(profanity_detection/normalizer.py)`

This stage exists specifically to satisfy the Product Vision's hardest requirement: catching "Fuuuuuck," "m@darchod," and similar disguised variants before any matching happens, so that Stage 5's dictionary lookup can stay a simple, explainable, admin-editable exact-match operation.

Normalization steps, applied in order:

1. **Unicode normalization (NFC)** — required for Devanagari input consistency (SRS-FR-DTCT-CFG-05); two visually-identical Devanagari strings with different combining-character encodings must normalize to the same form before comparison.
2. **Lowercasing** — Latin-script tokens only (case-folding does not apply to Devanagari).
3. **Repeated-character collapsing** — regex `(([a-z])\1{2,})` → replace runs of 3+ identical letters with a single instance, then re-check the dictionary; if no match, progressively try 2-repeat and 1-repeat collapses. This handles `(fuuuuuck → fuck)` (SRS-FR-PDE-02) without over-collapsing legitimate double letters (e.g., "book" is untouched since it only has a 2-run).
4. **Leetspeak / symbol substitution mapping** — a fixed substitution table `(@→a, 4→a, 3→e, 1→i, 0→o, $→s, 7→t)` applied to generate a normalized candidate string, e.g. `(m@darchod → madarchod)` (SRS-FR-PDE-04). Substitution is applied combinatorially only up to 2 substituted characters per word to bound the

search space and avoid false-positive explosions on legitimate words containing digits.

5. **Whitespace/punctuation stripping** at word boundaries, so that `f.u.c.k` or `f-u-c-k` still normalize to `fuck`.

Output: for every ASR word/n-gram, the normalizer emits a small ranked list of normalization candidates (original → fully normalized), each carried forward into Stage 5 with a `normalization_distance` score (0 = exact match, higher = more transformations applied). This distance later contributes negatively to the confidence score in Stage 7 — an exact dictionary hit is more trustworthy than a hit found only after three transformations.

Worked examples (traceable to Product Vision requirements):

Input	Normalization applied	Result
<code>Fuuuuuck</code>	lowercase → repeated-char collapse	<code>fuck</code>
<code>m@darchod</code>	leetspeak map (<code>@</code> → <code>a</code>)	<code>madarchod</code>
<code>bc</code> , <code>bkl</code> , <code>mc</code>	none (already normalized; matched as slang abbreviations directly in Stage 5)	<code>bc</code> , <code>bkl</code> , <code>mc</code>
<code>साले</code>	Unicode NFC normalization	<code>साले</code> (consistent encoding)

8. Stage 5 — Dictionary & Regex Matching

Module: `profanity_detection/dictionary_matcher.py`

Data source: `profanity_dictionary` table (Database Design §4.9), loaded into an in-memory hash set per language at worker startup and refreshed on a short TTL (e.g., 5 minutes) or on an explicit cache-bust signal when an admin edits the dictionary via `POST/PATCH/DELETE /admin/dictionary` — this is what makes SRS-FR-PDE-08 ("dictionary extensible without code deployment") actually true at runtime, not just at the database level.

Matching algorithm:

- For each normalized candidate from Stage 4, look up `normalized_term` in the in-memory set for the segment's detected language, and also in the `universal` language bucket (catches slang used across English/Hindi/Hinglish contexts, e.g. `bc`, `mc`, `bkl`).
- Sliding n-gram window (unigram, then bigram) is checked for multi-word entries (e.g., a curated phrase, not just single words).

3. On a hit, record: matched (`dictionary_id`), (`severity`), and (`detection_source = 'dictionary'`) (exact match) or (`'regex'`) (matched only via a pattern rule rather than an exact dictionary row — reserved for entries flagged as regex-type in the dictionary, e.g. a rule capturing any leetspeak variant of a root word without enumerating every combination).
4. Non-matches are passed to Stage 6.

This layer is intentionally the fastest and most explainable — it runs in-process against an in-memory hash set, contributing negligible latency to the overall $\leq 2\times$ -real-time budget, and every hit can be shown to the user with a precise "matched dictionary entry: X" explanation (SRS-FR-PDE-07), satisfying the explainability requirement discussed in the Tech Stack document's hybrid-architecture rationale.

9. Stage 6 — Context-Aware Scoring

Module: (`profanity_detection/context_scorer.py`)

Responsibility: Catch profanity that Stage 5 missed — novel slang, context-dependent terms, or heavily obfuscated variants beyond the normalizer's fixed rules — using semantic similarity rather than exact matching.

Approach (per Tech Stack §4.3): sentence-embedding similarity against a curated seed bank, not a fine-tuned classifier — this is the deliberate choice to avoid needing a labeled Hindi/Hinglish profanity training set that does not exist.

Algorithm:

1. Embed each transcript segment (not just the flagged word, but a small context window — the word plus 2-3 surrounding words) using a multilingual sentence-embedding model (MuRIL-based or a general multilingual sentence-transformer, per the Tech Stack recommendation).
2. Compute cosine similarity between the segment embedding and every entry in (`data/embeddings/seed_profanity_examples.json`) (curated example phrases per language, expandable by admins without retraining — this is the concrete mechanism behind the Product Vision's "support future learning" requirement).
3. If max similarity exceeds a configurable threshold (default (`0.72`), tuned empirically against the internal test set referenced in PRD §4.6/AC-1), emit a candidate detection with (`detection_source = 'model_inferred'`) and a raw similarity score.
4. This stage also **re-scores** Stage 5 dictionary hits that occurred in an ambiguous context (e.g., a word that is profane in one context but benign in another, per the PRD Edge Case table) — the embedding similarity against context-specific seed examples can *lower* an otherwise-high dictionary-match confidence, which is then finalized in Stage 7.

Because this stage runs an embedding model per segment rather than per word, and multilingual sentence-transformer models are small relative to the ASR model already loaded, this stage adds a bounded, predictable amount of CPU time and does not require a network call (no cloud LLM dependency), preserving the offline/cost-free constraint from the SRS.

10. Stage 7 — Confidence Aggregation

Module: `profanity_detection/confidence_aggregator.py`

Responsibility: Produce the final `confidence_score` (0.0–1.0) and `detection_source` persisted per row in `detections` (Database Design §4.8), satisfying SRS-FR-PDE-05.

Aggregation formula:

```
base_score =
    0.95 if detection_source == 'dictionary' (exact match,
normalization_distance == 0)
    0.85 if detection_source == 'dictionary' (matched only after
normalization, distance > 0)
    0.75 if detection_source == 'regex'
    similarity_score (0.0–1.0) if detection_source == 'model_inferred'

final_score = clamp(base_score - (0.03 * normalization_distance) +
context_adjustment, 0.0, 1.0)
```

Where `context_adjustment` is the signed delta produced by Stage 6 re-scoring (Section 9, step 4) — positive if the surrounding context reinforces the profane reading, negative if it suggests a benign homonym use.

Deduplication: if the same time range is flagged by both Stage 5 and Stage 6 independently, the two candidate detections are merged into a single `detections` row rather than persisted as duplicates — this is the same merge/dedupe behavior already specified for Combined detection mode at the user-facing level (SRS-FR-DTCT-CFG-03), applied here internally between the dictionary and model layers.

Threshold filtering: All candidate detections are persisted regardless of score (nothing is silently dropped, since the user can filter by confidence in the Timeline Editor per SRS-FR-PDE-06/FR-TLE — dropping low-confidence candidates at the pipeline level would remove information the human reviewer is specifically there to adjudicate). A configurable `MIN_PERSIST_CONFIDENCE` (default `0.3`) exists purely to discard obvious noise from the embedding layer, not to enforce the product-level precision/recall target — that target (SRS-FR-PDE AC-1: recall $\geq 85\%$, precision $\geq 80\%$) is measured and tuned against the *default-displayed* threshold in the UI, not against what's stored.

11. Detection Instance Output Contract

Each finalized detection maps directly onto the `detections` table schema (Database Design §4.8) and the API response shape already defined in API Design §5.1:

json

```
{
  "detected_text": "bakchodi",
  "language": "hindi",
  "start_time": 12.450,
  "end_time": 13.100,
  "confidence_score": 0.94,
  "detection_source": "dictionary",
  "dictionary_id": "uuid-or-null",
  "replacement_mode": "beep",
  "is_user_added": false,
  "is_dismissed": false
}
```

`replacement_mode` at creation time is inherited from the file's `detection_configs.replacement_mode` (the mode chosen at upload time); it is only overridden per-instance later if the user changes it individually in the Timeline Editor (FR-TLE-05).

12. Stage 8 — Audio Editing

Module: `audio_processing/audio_editor.py`

Responsibility: For every non-dismissed detection, replace the corresponding audio segment according to its `replacement_mode`, satisfying SRS-FR-REPL-01/02/03/05.

Interface:

python

```
class AudioEditor:
    def apply_replacements(
        self,
        audio_path: str,
        detections: list[DetectionInstance],
        replacement_assets: ReplacementAssets,
    ) -> str:
        """Returns path to the edited audio file."""
```

Algorithm (PyDub-based):

1. Load the extracted audio (`AudioSegment.from_wav(audio_path)`).
2. Sort detections by `start_time` ascending.
3. For each detection, apply a **safety padding margin** of 100–150ms on both boundaries (per the PRD Edge Case table: "profane word at very start/end of a clip"), clamped to not overlap the previous/next detection's own padded range.
4. Per `replacement_mode`:
 - **Beep**: generate a sine-wave tone (`pydub.generators.Sine(frequency=1000)`), configurable frequency per SRS-FR-REPL-01) with duration exactly matching the padded segment length, and overlay/replace it in place.
 - **Mute**: overlay `AudioSegment.silent(duration=segment_length_ms)` in place of the original segment.
 - **Custom sound**: load the user-uploaded clip from `custom_sound_path`; if shorter than the segment, loop it; if longer, trim it — always matching the original segment's duration within the ± 50 ms tolerance specified in SRS-FR-REPL-05, which is the concrete numeric target this trim/loop logic is built to satisfy.
5. Reassemble the full audio track by concatenating: [untouched audio before first detection] + [replaced segments and untouched gaps between them, in order] + [untouched audio after last detection].
6. Export the reassembled track to a working WAV file for Stage 9.

Regeneration efficiency note (ties back to PRD Open Question §11): because this stage operates purely on the already-extracted audio plus the current `detections` rows, a "Regenerate" action (FR-TLE-08) does **not** need to re-run Stages 1–7 (extraction, ASR, detection) — only Stage 8–9 re-run, using the edited detection set. This is what makes the AC-2 requirement in PRD §4.7 ("regeneration only reprocesses affected segments, not a full re-transcription") achievable, and resolves the open question raised in the PRD as "full file vs. edited segments only" — the answer, given this pipeline design, is: full audio re-assembly (cheap, since it's just PyDub segment operations) but zero re-transcription (expensive, and unnecessary since the transcript/timestamps didn't change).

13. Stage 9 — Media Muxing

Module: `audio_processing/media_muxer.py`

Responsibility: Re-combine the edited audio track with the **original, untouched video stream**, preserving resolution, frame rate, and metadata (Product Vision constraint; SRS-FR-EXPT-01/AC-1).

Algorithm (FFmpeg):

```
ffmpeg -i {original_video_path} -i {edited_audio_path} \  
-map 0:v -map 1:a \  
-c:v copy \  
-c:a aac -b:a 192k \  
-map_metadata 0 \  
-shortest \  
{output_video_path}
```

Key decisions encoded in this command:

- `-c:v copy` — the video stream is **stream-copied**, never re-encoded, which is the only way to guarantee zero quality/resolution/frame-rate loss (this is the specific technical mechanism behind the "preserve video quality/frame rate/resolution" requirement stated repeatedly across the Product Vision, PRD, and SRS).
- `-map_metadata 0` — pulls container metadata from the original file, not the edited audio, preserving original metadata as required.
- Audio is re-encoded (not stream-copied) because PyDub's edits produced a modified PCM/WAV stream that must be re-compressed into the target container's expected audio codec (typically AAC for MP4 containers).
- `-shortest` guards against a duration mismatch between video and reassembled audio causing trailing silence/frozen frames; combined with the $\pm 50\text{ms}$ tolerance enforced in Stage 8, drift stays within the $<100\text{ms}$ sync budget from the Product Vision's success metrics and SRS-FR-EXPT AC-2.

Audio-only files (mp3/wav/etc. uploads) skip this stage entirely — the Stage 8 output is the final audio export, simply transcoded to the original format via a lighter FFmpeg pass if needed.

14. Pipeline Orchestration

Module: `pipeline/processing_pipeline.py`

```
python
```

```

class ProcessingPipeline:
    def run_initial(self, file_id: str, config: DetectionConfig) -> PipelineResult:
        audio_path = self.extractor.extract(...)
        transcription = self.asr.transcribe(audio_path)
        segments = self.language_detector.tag(transcription)
        candidates = []
        for segment in segments:
            normalized = self.normalizer.normalize(segment.words)
            dict_hits = self.dictionary_matcher.match(normalized, segment.language)
            model_hits = self.context_scorer.score(segment, dict_hits)
            candidates += self.aggregator.aggregate(dict_hits, model_hits)
        detections = self.merge_with_custom_words(candidates, config)
        self.repository.save_detections(file_id, detections)
        self.audio_editor_and_mux(file_id, detections) # first-pass render
        return PipelineResult(status="READY_FOR_REVIEW", detections=detections)

    def run_regeneration(self, file_id: str) -> PipelineResult:
        detections = self.repository.get_active_detections(file_id) # excludes :
        edited_audio = self.audio_editor.apply_replacements(
            self.repository.get_extracted_audio_path(file_id), detections, ...
        )
        output_path = self.media_muxer.mux(
            self.repository.get_original_video_path(file_id), edited_audio
        )
        self.repository.save_export(file_id, output_path)
        return PipelineResult(status="COMPLETED")

```

Idempotency (SRS-NFR-REL-02, Architecture §1.2): every stage writes its output to a content-addressed or job-scoped working path (e.g., keyed by `(job_id)`), never mutating the original upload in place. A retried job simply re-runs from Stage 1 against the same immutable input and overwrites its own prior (also-immutable-to-others) intermediate outputs — no partial state from a failed attempt can corrupt a later successful attempt, satisfying the "idempotent, resumable jobs" principle stated in the Architecture document.

Custom/Combined detection modes: `(merge_with_custom_words)` (called in `(run_initial)`) is where the Detection Config's `(mode)` (`(automatic)`/`(custom)`/`(combined)`) actually takes effect — custom words from `(detection_configs.custom_words)` are matched using the *same* Stage 4–5 normalization and dictionary-matching code path (a custom word list is just an ephemeral, per-file dictionary), then merged and deduplicated against the automatic-detection candidates exactly as SRS-FR-DTCT-CFG-03 specifies, reusing infrastructure rather than introducing a parallel matching implementation.

15. Performance Considerations

Concern	Mitigation
ASR is the dominant cost (SRS-NFR-PERF-01: $\leq 2\times$ real-time)	<code>int8</code> quantized Faster-Whisper on CPU; GPU used opportunistically when available (Tech Stack §3.3, point 5) without being a hard dependency
Long files (3-hour lecture, PRD Edge Case)	Chunked transcription with incremental progress published via the WebSocket status channel (Architecture §4.1) after each chunk, not only at full completion
Concurrent jobs (SRS-NFR-PERF-04)	Each pipeline stage is stateless per invocation; horizontal scaling is achieved by running more worker replicas against the same Redis queue (Architecture §6.1), not by parallelizing within a single job
Repeated regeneration cost	As established in §12, regeneration skips Stages 1–7 entirely — this is the single largest performance lever available for the Timeline Editor's "Regenerate" action

16. Error Handling & Retry

Each stage raises a stage-specific exception (`AudioExtractionError`, `TranscriptionError`, `MuxingError`, etc.) caught at the `processing_task.py` Celery task boundary (Folder Structure §4), which:

- Writes a `job_logs` row with `stage`, `level = 'ERROR'`, and the exception message (Database Design §4.7).
- Increments `jobs.retry_count` and re-enqueues with exponential backoff if under `max_retries`, or marks the job `FAILED` and publishes a `job_failed` WebSocket event — this is precisely the sequence already diagrammed in System Architecture §4.3.
- For the specific "music only / no speech detected" edge case, an empty-but-valid transcription is **not** treated as an error — the pipeline completes normally with zero detections and the file status becomes `READY_FOR_REVIEW`, matching the PRD Edge Case table's expected behavior.

17. Extensibility Points (Concrete Realization of Architecture §7.3)

Interface	Current implementation	Future swap example
<code>SpeechRecognitionService</code>	<code>FasterWhisperEngine</code>	Cloud ASR API (Google/Azure) for a future paid tier
<code>forced_alignment.py</code>	Pass-through (Faster-Whisper's native word timestamps)	Dedicated Indic wav2vec2 alignment model
<code>ProfanityDetectionService</code>	Normalize → Dictionary/Regex → Embedding scorer	Fine-tuned IndicBERT/MuRIL classifier once labeled data exists
<code>AudioEditor</code> replacement strategy	Beep / Mute / Custom sound	<code>VoiceReplacementStrategy</code> (AI voice cloning, Future Scope) plugged in as a fourth <code>replacement_mode</code> without <code>apply_replacements</code>

18. Testing & Evaluation Strategy

- **Unit tests** (`ai-pipeline/tests/`): `test_normalizer.py` asserts the exact worked examples from Section 7 (e.g., `Fuuuuuck → fuck`, `m@darchod → madarchod`); `test_dictionary_matcher.py` and `test_context_scorer.py` test against fixture transcripts with known expected detections.
- **Benchmark scripts** (`scripts/benchmark_asr.py`, `scripts/benchmark_detection.py`, per Folder Structure §10): run against a held-out labeled test set to produce the recall/precision numbers referenced in PRD §4.6/AC-1 (recall ≥85%, precision ≥80%) and SRS §5.4 language-detection accuracy (≥85%).
- **Integration test** (`test_pipeline_integration.py`): runs the full Stage 1-9 pipeline against `scripts/generate_test_media.py` fixtures (short synthetic clips with known profanity at known timestamps) and asserts both correct detection and <100ms audio/video sync drift in the final muxed output.

19. Configuration

All tunable values in this document are externalized (per SRS-NFR-MAINT-03), not hardcoded, via `ai-pipeline/sentinelcut_ai/pipeline/pipeline_config.py` and environment variables consumed at worker startup:

Config key	Default	Section
ASR_MODEL_SIZE	medium	§5
ASR_COMPUTE_TYPE	int8	§5
CHUNK_LENGTH_SECONDS	1800 (30 min)	§5
CONTEXT_SIMILARITY_THRESHOLD	0.72	§9
MIN_PERSIST_CONFIDENCE	0.3	§10
BOUNDARY_PADDING_MS	125	§12
BEEP_FREQUENCY_HZ	1000	§12
MAX_JOB_RETRIES	3	§16 (mirrors jobs.max_retries default in Database Design)

20. Next Document

Document 10: UI/UX Design — screen-level design specifications (Landing Page, Dashboard, Upload, Timeline Editor, Admin Dashboard, Light/Dark mode) for the Next.js frontend, consuming the API contract from Document 8 and reflecting the detection/confidence data shapes defined in this AI Pipeline document (in particular, how per-word confidence scores and detection sources from Section 11 are visualized in the Timeline Editor).

End of AI Pipeline Document